

1 Exercice : Comparaison d'implémentation de la mise-à-jour des mailles fantômes à l'aide de communications point-à-point et one-sided

Dans cette exercice, nous allons nous focaliser sur la mise-à-jour des mailles fantômes dans le code du jeu de la vie. Nous avons vu lors du TD précédent le principe de fonctionnement et implémenté un mécanisme de protection/reprise. Pour rappel, **le domaine est découpé par bandes, ce qui fait que chaque processus a simplement un voisin au-dessus et un en-dessous**. Nous allons comparer trois implémentations :

- Utilisation classique des communications point-à-point
- Utilisation de communications one-sided avec des synchronisations collectives (MPI_Win_fence)
- Utilisation de communications one-sided avec le mécanisme minimal (post-start-complete-wait)

Les deux modes de synchronisations comparés dans ce TD correspondent à des communications de type "Active Target". En effet, le processus cible (qui détient la fenêtre où seront écrites les données) est actif dans la phase de communication : il appartient au groupe de processus dans l'appel collectif à MPI_Win_fence d'une part, et il expose explicitement la fenêtre (MPI_Win_post et MPI_Win_wait) d'autre part.

Informations diverses :

- Il est conseillé de charger un module MPI récent :
 > module avail mpi
 > module load mpi/...
- La commande > make vous permet de compiler le code.
- Le code se lance de la manière suivante :
 > srun -n <nb_processes> mlife-<executable> -x <dim_x> -y <dim_y> -i <nb_iter>
- exemple d'essai fonctionnel avec les communication point-à-point :
 > srun -n 4 mlife-pt2pt -x 50 -y 50 -i 20
- exemple d'essai fonctionnel avec les communications one-sided fence :
 > srun -n 4 mlife-fence -x 50 -y 50 -i 20
- exemple d'essai fonctionnel avec les communication one-sided pscw :
 > srun -n 4 mlife-pscw -x 50 -y 50 -i 20

Question1

[mlife-pt2pt.c] Nous commençons ici avec l'implémentation classique à base de communications point-à-point à définir dans le fichier mlife-pt2pt.c.

- Duplication et libération du communicateur dédié à cet échange.
- Création de la communication point-à-point non-bloquante.

Question2

[mlife-fence.c] Nous allons maintenant utiliser les communications one-sided avec le mécanisme de synchronisation à l'aide de clôtures (fence). Ces communications sont à définir dans le fichier mlife-fence.c.

- Création et libération de la fenêtre (MPI_Win_create, MPI_Win_free)

- Délimitation des phases d'accès à la fenêtre mémoire à l'aide d'une barrière de synchronisation (MPI_Win_fence). Ne pas oublier de définir les options passées à l'ouverture et à la fermeture.
- Ecriture des données dans les processus voisins (MPI_Put)

Question3

[mlife-pscw.c] Nous allons maintenant utiliser les communications one-sided avec le mécanisme de synchronisation minimal pscw. Ces communications sont à définir dans le fichier `mlife-pscw.c`.

- Création et libération de la fenêtre mémoire (MPI_Win_create, MPI_Win_free).
- Mise en place du mécanisme pscw (MPI_Win_post, MPI_Win_start, MPI_Win_complete, MPI_Win_wait)
- Ecriture des données dans les processus voisins (MPI_Put)